Assignment 1

Week 5, hand-in Friday 18h00 of Week 6

Object Oriented Programming

Software Workshop I

Author: Jacqui Chetty

Marker (automated): Pieter Joubert

Assessed assignment 1: 100 marks, 35% of the module mark

Rules:

1. Throughout the assignment we refer to **attributes** although these can be called data fields, instance variables.
2. For each class refer to it's corresponding test to verify field and method naming conventions.
3. Unless stipulated all attributes are private and all methods are public.
4. Although there are many ways to construct an application, you are required to adhere to the rules as stipulated below (to achieve marks).
5. If variable names are not stipulated you can use your own names for variables, this shows that you have written the application (we will check for plagiarism).
6. You are not allowed to make use of regular expressions and / or make use of any classes such as Pattern to determine the String expression (for question 2).
7. Only the imports that we request you to provide are allowed, NO OTHER imports can be included.

Question 1: [60 marks]
Description of the assignment:
You are tasked with creating a Java application, given the template provided. Your task is to create a customer application, where a customer can order books. Once an order has been placed, the order needs to be shipped. Therefore, a shipping cost must be determined. Once complete, an invoice for the order, including the shipping cost, needs to be generated. For example, a customer can buy a book; and the book shop will then place the order. The shipping for the order must be calculated and a shipping order generated. Finally, an invoice must be produced and all invoices added to a repository. The repository can be used to search for an invoice. Further detail regarding each class is now discussed.

Customer class: [**5 marks**] - the Customer class consists of customer objects

- The `attributes` are as follows: name, phone and eMailAddress (all String);
- The `Customer()` constructor: pass in 3 variables (name, phone, eMailAddress) as parameters to the constructor;
- The `getter()` methods: each attribute must be provided with a getter();

Stock class: [**5 marks**] - the Stock class consists of book objects

- The `attributes` are as follows: bookName and author (both String), and a price (double);
- The `Stock()` constructor: pass in 3 variables (bookName, author, price) as parameters to the constructor;
- The `getter()` methods: each attribute must be provided with a getter();

Order class: [**5 marks**] - the Order class consists of order objects

- The `attributes` are as follows: customer (object of Customer class) and a book i.e. stock (object of Stock class);
- The `Order()` constructor: pass a customer object and a stock object to the constructor;
- The `getter()` method: each attribute must be provided with a getter();

Shipping class: [**15 marks**] - the Shipping class sets the shipping cost and creates a shipping order

- `import`: import the LocalDate from the appropriate package;
- The `attributes` are as follows: an order object (Order) , a shipDate (LocalDate), a shipCost (double) and a public static variable called countUrgent (int);
- The `Shipping()` constructor: pass an order object and a shipDate to the constructor;
- The `getter()` methods: the attributes shipDate and shipCost require a getter();
- The `setShipCost()` method: pass the shipCost in as a parameter, set the parameter to the attribute shipCost;
- The `calcShipCost()` method: the header is as follows - public double calcShipCost(boolean isUrgent) - pass a boolean parameter isUrgent to the method, to determine the shipCost make use of the isUrgent value; depending on whether the shipping is urgent or not the shipping cost will differ, urgent shipping means that the shipCost is set to 5.45, otherwise if it is not urgent set the shipCost to 3.95, if the shipping is urgent increment the countUrgent, return the shipCost;

Invoice class: [10 marks] - the Invoice class generates an invoice for the order placed

- The `attributes` are as follows: invoiceNbr (String), the book order, i.e. stock (Stock) object, shipOrder (Shipping) object and the totalCost (double);
- The `Invoice()` constructor: pass an invoiceNbr, stock object, and the shipOrder object to the constructor;
- The `getter()` method: the attribute invoiceNbr requires a getter();
- The `invoice()` method: the header is as follows - public double invoice() - calculate the totalCost for shipping this order which would be the price of the book (stock) and the shipping cost, return the totalCost;

BookStore class: [15 marks] - create an ArrayList<> (repository) to keep all invoices as well as searching for an invoice

- `import`: import the ArrayList from the .util package;
- The `attributes` are as follows: declare the ArrayList<Invoice> invoices, final String message is provided to you;
- The `BookStore` constructor: complete the declaration of the ArrayList<> invoices (instantiate);
- The `getter()` method: getInvoices();
- The `searchOrder()` method: the header is as follows - public Invoice searchOrder(String invoiceNbr) - pass an invoiceNbr to be searched for, provide the necessary code to complete the search, if the invoice number is found in the ArrayList return the invoice object, otherwise return a null value;
- The `pilingUpOfOrders()` method: the header is as follows - public String pilingUpOfOrders() - verify if the countUrgent is greater than 5, if this is the case return the attribute message otherwise return null;

BookStoreMain class: 5 marks - the main() is found here and this is where the application is executed from

- The `imports`: you are required to import the java.time (LocalDate) packages;
- The `main()` method: (declare customer as customer1, stock1, order1, etc as these steps are repeated for customer2, etc)
    1. create a bookStore object and instantiate the object;
    2. instantiate an object customer for the Customer class so that a customer can buy a book(s) - see unit test for more detail;

3. instantiate an object book on Stock so that a book can be chosen to buy - see unit test for more detail;
4. instantiate an object order on Order to place the order - see unit test for more detail;
5. declare and assign values for year, month and day (local variables, int) and pass these values to the shipDate using LocalDate;
6. instantiate an object shipOrder on Shipping passing the order object and shipDate to the constructor in Shipping;
7. call calcShipCost() passing isUrgent for shipOrder and calculate the shipCost (declare it as a double);
8. instantiate an invoice object from the Invoice class and pass an invoiceNbr, book object and the shipOrder object to the Invoice class through the constructor;
9. call the invoice() to cost the invoice for the invoice object;
10. add the invoice object to the ArrayList invoices;
11. call pilingUpOfOrders() to verify if the urgent orders to be shipped are becoming too much (created in BookStore class);
12. repeat steps 2 to 11 to create a second ordering of books with a new customer, calculating all necessary costs;

- instantiate an object foundInvoice for Invoice and call the searchOrder() method passing the invoice number for invoiceNbr1 as an argument to the method;

Q2 IS ON THE NEXT PAGE

Question 2: [40 marks]
Description of the assignment:
CalculatorMain is provided to you, however you need to create the other class, namely Calculator class yourself. For this exercise you will create a Calculator class and add some basic functionality to it. The Basic Calculator is able to evaluate simple arithmetical expressions involving integer operands and the operators: * , / , + , -.

The functionality of the basic calculator is as follows.

1. Evaluating simple arithmetical expressions

2. The calculator can evaluate expressions of the following form: operand[space]operator[space]operand For example, 3 + 4, 2 * 10, 10 / 5, 39 - 47.

3. Thus, the operands are any valid integer and the operator is one of * , / , + or -. There is a space character between each operand and the operator.

4. For the Basic Calculator, if the user enters an expression (as a String) that is not precisely of the form shown above then the program should return a boolean value.

5. If the user enters an expression involving a 'divide-by-zero' error then the program should also output a boolean value in that case too.

   Calculator class: [40 marks]

   (a) You must create a class called Calculator;

   (b) The mandatory attributes are operator (char), operand1 and operand2 (int) and expression (String);

   (c) The `constructor`:public Calculator() is the default Constructor;

   (d) The `getter()` methods: getter() for each mandatory attribute;

   (e) The `setExpression()` method: pass the expression, i.e. the arithmetic expression to be evaluated) to the method, and assign it to the class attribute;

   (f) The `verify()` method: the header is as follows - public boolean verify()

      i. This method determines whether an expression is a valid expression or not and returns either true or false;

      ii. You may call another method from this method or you may find you don't need to;

      iii. You need to test whether there is a valid operator, a space either side of the operator, that both operands are valid integer numbers (only integers are used for this application), and that there is no division by zero;

      iv. Only if all tests pass can a true be returned, otherwise the expression cannot be

evaluated as it is not a valid expression;

v. Your operator as well as both operands MUST be assigned / determined as part of this method;

6. The `evaluate()` method: the header is as follows - public int evaluate()

   (a) This method evaluates the expression and returns the result as a int;